PROJECT DOCUMENTATION

Music Playlist & Listening-Stats Manager

Phase 2 Progress Report

Work completed so far and next steps

Project	Music Playlist & Listening-Stats Manager
Prepared by	Yasashwini — 2620030206 Janani — 2620040056
Supervisor	Rakesh.K
Report stage	Review 2 — Progress

Prepared in simple student-friendly language based on the Java code shared for this project. Update the report if the implementation changes or your faculty provides a required format.

1. Project Progress

Several Java class examples have been written for the Music Playlist & Listening-Stats Manager. They demonstrate track objects, keyboard input, control structures, overloaded methods, array storage, and inheritance. The examples still need to be connected and tested together.

2. Classes Written

• Track: stores track details and includes a constructor and a small main example.

• InputHandler: reads a song title and duration using Scanner.

• PlaylistController: demonstrates switch, if-else, for, and do-while.

• SongManager: demonstrates adding a song with and without a genre argument.

• PlaylistStorage: stores Track objects in a fixed-size array and displays them.

• Song subclass example: demonstrates inheritance and overriding displayInfo().

3. Java Concepts Practised

Concept	Where it appears	Purpose
Class and object	Track, Song, myTrack	Represent songs as objects.
Constructor	Track(...) and Song(...)	Set initial values.
Scanner	InputHandler	Read keyboard input.
Conditions and switch	PlaylistController	Select actions and classify duration.
Loops	for and do-while	Repeat instructions.
Method overloading	SongManager.addSong(...)	Same method name with different parameters.
Array of objects	PlaylistStorage	Store several Track objects.
Inheritance/overriding	Song extends Track	Reuse track details and customise display.

4. Checks Needed

• Compile each class separately and correct errors.

• Avoid duplicate Track class definitions in the same package.

• Update the do-while loop so the choice can change and the loop can exit.

• Check array capacity before storing a new track.

• Pass the input values to the playlist storage or return them from InputHandler.

• Run test cases and record actual results.

5. Current Limitations

The supplied code contains separate examples rather than one complete application. Some entered data is not passed to the storage class. The menu is not fully connected, and test results have not yet been provided.

6. Next Steps

• Keep one Track definition in the final application.

- Connect the classes through a main menu.
- Add and display tracks safely.
- Add input validation and test each option.
- Capture actual console output for the final report.

7. Progress Conclusion

The code shows progress in learning core Java concepts. The next task is to integrate the separate examples into one working playlist application.